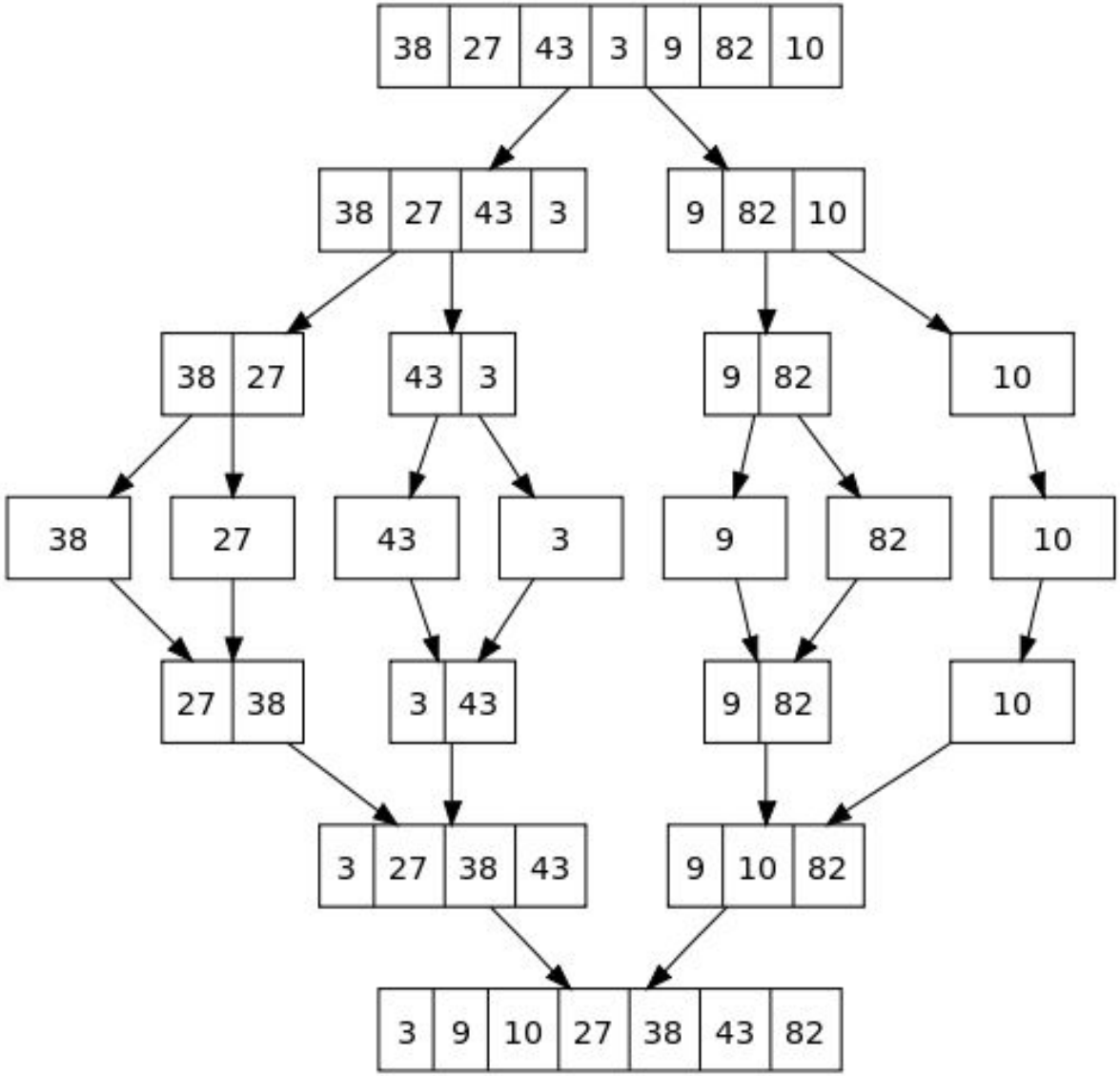
Algoritmos Recursivos (Cont)

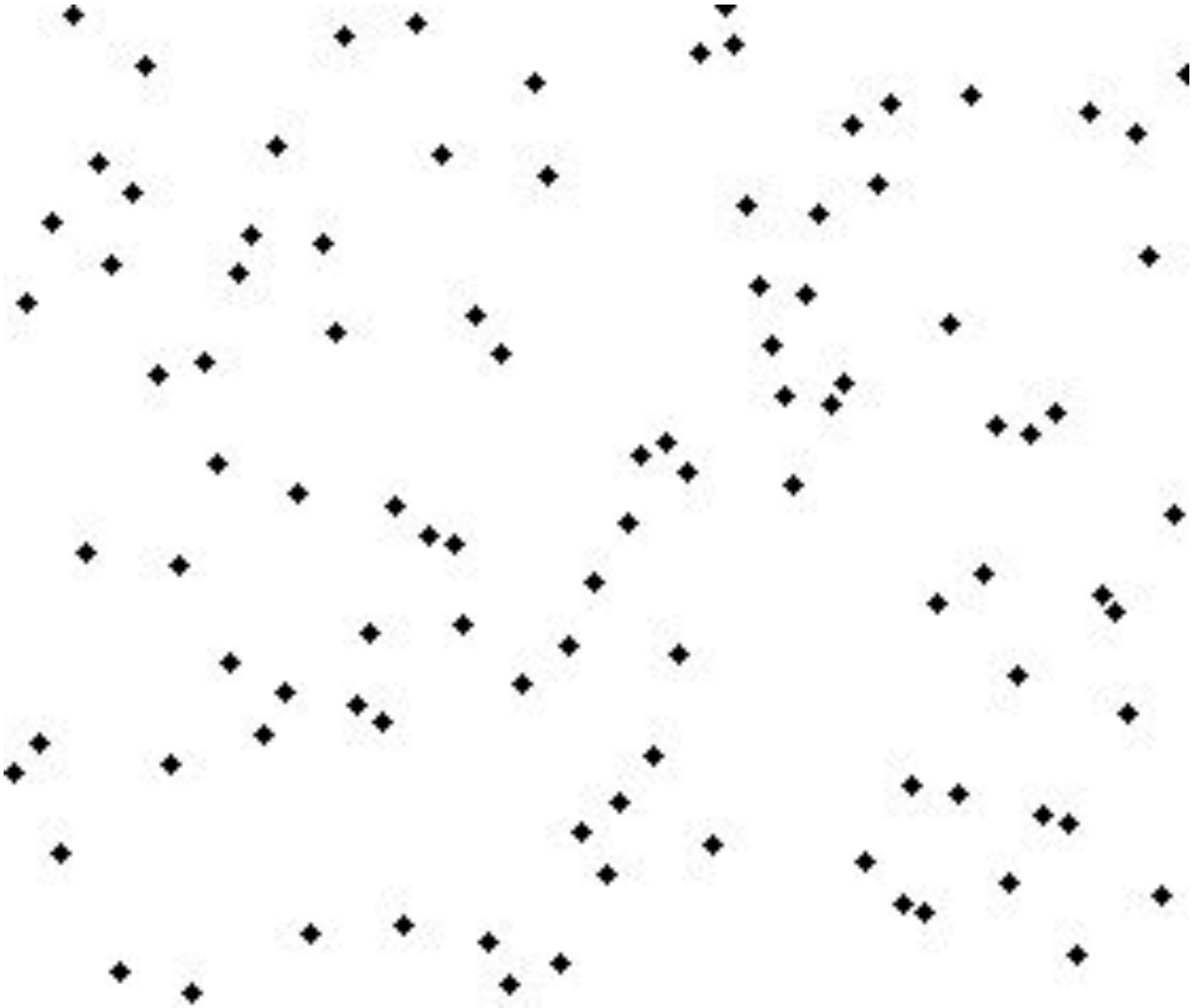
Recursividade

- **Quando utilizar recursividade ?**
  - Quando o tamanho do problema for muito grande;
- **Por que utilizar recursividade ?**
  - Melhorar a clareza e simplicidade do código
  - Reduzir a instância do problema em uma instância menor

Exemplo1 – MergeSort



Exemplo2 - MergeSort



Algoritmos Recursivos

- **Vantagens**
  - Eliminação de controle sobre variáveis
  - Código mais limpo e conciso
  - Simplificação do problema em subproblemas

Algoritmos Recursivos

- **Desvantagens**
  - Geralmente são mais lentos
  - Ocupam bastante memória
  - Possibilidade de loops infinitos

Métodos

- **Para obtenção de limites assintóticos O e a função de complexidade theta :**
  - Método de substituição
  - Método de Árvore de recursão
  - Método Mestre

Algoritmos Recursivos

- **Método de substituição**
  - Pressupor a forma da solução
  - Usar a indução matemática para encontrar as constantes e mostrar que a solução funciona.

Algoritmos Recursivos

- **Método de substituição – Exemplo**

Defina um limite superior sobre a recorrência abaixo:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n > 1. \end{cases} \quad (4.2)$$

As condições limite representam outra classe de detalhes que em geral ignoramos. Tendo em vista que o tempo de execução de um algoritmo sobre uma entrada de dimensões constantes é uma constante, as recorrências que surgem dos tempos de execução de algoritmos geralmente têm $T(n) = \Theta(1)$ para n suficientemente pequeno. Em consequência disso, por conveniência, em geral omitimos declarações sobre as condições limite de recorrências e iremos supor que $T(n)$ é constante para n pequeno. Por exemplo, normalmente enunciamos a recorrência (4.1) como

$$T(n) = 2T(n/2) + \Theta(n) , \quad (4.3)$$

sem fornecer explicitamente valores para n pequeno. A razão é que, embora a mudança no valor de $T(1)$ altere a solução para a recorrência, normalmente a solução não muda por mais de um fator constante, e assim a ordem de crescimento não é alterada.

Quando enunciamos e resolvemos recorrências, com freqüência omitimos pisos, tetos e condições limite. Seguimos em frente sem esses detalhes, e mais tarde definimos se eles têm importância ou não. Em geral, eles não têm importância, mas é importante saber quando têm. A experiência ajuda, e também alguns teoremas declarando que esses detalhes não afetam os limites assintóticos de muitas recorrências encontradas na análise de algoritmos (ver Teorema 4.1). Porém, neste capítulo, examinaremos alguns desses detalhes para mostrar os últimos métodos de solução de pontos de recorrência.

4.1 O método de substituição

O método de substituição para resolver recorrências envolve duas etapas:

1. Pressupor a forma da solução.
2. Usar a indução matemática para encontrar as constantes e mostrar que a solução funciona.

O nome vem da substituição da resposta pressuposta para a função quando a hipótese indutiva é aplicada a valores menores. Esse método é eficiente, mas obviamente só pode ser aplicado em casos nos quais é fácil pressupor a forma da resposta.

O método de substituição pode ser usado para estabelecer limites superiores ou inferiores sobre uma recorrência. Como exemplo, vamos determinar um limite superior sobre a recorrência

$$T(n) = 2T(\lfloor n/2 \rfloor) + n , \quad (4.4)$$

Algoritmos Recursivos

- **Método de substituição – Exemplo**

Defina um limite superior sobre a recorrência abaixo:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n > 1. \end{cases} \quad (4.2)$$

As condições limite representam uma classe de detalhes que em geral ignoramos. Tendo em vista que o tempo de execução de um algoritmo sobre uma entrada de dimensões constantes é uma constante, as recorrências que surgem dos tempos de execução de algoritmos geralmente têm $T(n) = \Theta(1)$ para n suficientemente pequeno. Em consequência disso, por conveniência, em geral omitiremos declarações sobre as condições limite de recorrências e iremos supor que $T(n)$ é constante para n pequeno. Por exemplo, normalmente enunciamos a recorrência (4.1) como

$$T(n) = 2T(n/2) + \Theta(n) , \quad (4.3)$$

sem fornecer explicitamente valores para n pequenos. A razão é que, embora a mudança no valor de $T(1)$ altere a solução para a recorrência, normalmente a solução não muda por mais de um fator constante, e assim a ordem de crescimento não é alterada.

Quando enunciamos e resolvemos recorrências, com frequência omitimos pisos, tetos e condições limite. Seguimos em frente sem esses detalhes, e mais tarde definimos se eles têm importância ou não. Em geral, eles não têm importância, mas é importante saber quando têm. A experiência ajuda, e também alguns teoremas declarando que esses detalhes não afetam os limites assintóticos de muitas recorrências encontradas na análise de algoritmos (ver Teorema 4.1). Porém, neste capítulo, examinaremos alguns desses detalhes para mostrar os últimos métodos de solução de limites de recorrência.

4.1 O método de substituição

O método de substituição para resolver recorrências envolve duas etapas:

1. Pressupor a forma da solução.
2. Usar a indução matemática para encontrar as constantes e mostrar que a solução funciona.

O nome vem da substituição da resposta pressuposta para a função quando a hipótese indutiva é aplicada a valores menores. Esse método é eficiente, mas obviamente só pode ser aplicado em casos nos quais é fácil pressupor a forma da resposta.

O método de substituição pode ser usado para estabelecer limites superiores ou inferiores sobre uma recorrência. Como exemplo, vamos determinar um limite superior sobre a recorrência

$$T(n) = 2T(\lfloor n/2 \rfloor) + n , \quad (4.4)$$

Passo 1: Suponha que a solução seja T(n) = O(n lg n).

Logo T(n) <= c.n lg n

Algoritmos Recursivos

- **Método de substituição – Exemplo**

Defina um limite superior sobre a recorrência abaixo:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1, \\ 7T(\lceil n/2 \rceil) + 7T(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n > 1. \end{cases} \quad (4.2)$$

As condições limite representam uma classe de detalhes que em geral ignoramos. Tendo em vista que o tempo de execução de um algoritmo sobre uma entrada de dimensões constantes é uma constante, as recorrências que surgem dos tempos de execução de algoritmos geralmente têm $T(n) = \Theta(1)$ para n suficientemente pequeno. Em consequência disso, por conveniência, em geral omitiremos declarações sobre as condições limite de recorrências e iremos supor que $T(n)$ é constante para n pequeno. Por exemplo, normalmente enunciamos a recorrência (4.1) como

$$T(n) = 2T(n/2) + \Theta(n) , \quad (4.3)$$

sem fornecer explicitamente valores para n pequenos. A razão é que, embora a mudança no valor de $T(1)$ altere a solução para a recorrência, normalmente a solução não muda por mais de um fator constante, e assim a ordem de crescimento não é alterada.

Quando enunciamos e resolvemos recorrências, com frequência omitimos pisos, tetos e condições limite. Seguimos em frente sem esses detalhes, e mais tarde definimos se eles têm importância ou não. Em geral, eles não têm importância, mas é importante saber quando têm. A experiência ajuda, e também alguns teoremas declarando que esses detalhes não afetam os limites assintóticos de muitas recorrências encontradas na análise de algoritmos (ver Teorema 4.1). Porém, neste capítulo, examinaremos alguns desses detalhes para mostrar os últimos métodos de solução de limites de recorrência.

4.1 O método de substituição

O método de substituição para resolver recorrências envolve duas etapas:

1. Pressupor a forma da solução.
2. Usar a indução matemática para encontrar as constantes e mostrar que a solução funciona.

O nome vem da substituição da resposta pressuposta para a função quando a hipótese indutiva é aplicada a valores menores. Esse método é eficiente, mas obviamente só pode ser aplicado em casos nos quais é fácil pressupor a forma da resposta.

O método de substituição pode ser usado para estabelecer limites superiores ou inferiores sobre uma recorrência. Como exemplo, vamos determinar um limite superior para a recorrência

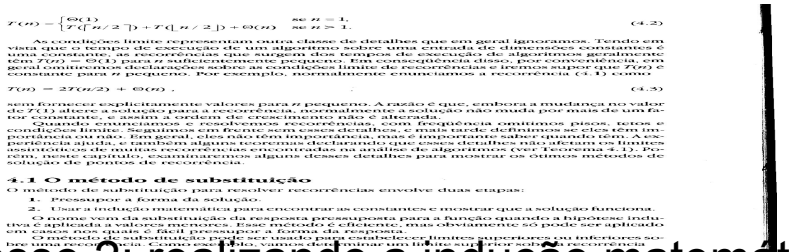
Provando a suposição: T(n) <= c. n lg n

$$\begin{aligned} T(n) &\le 2(c\,\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\le cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\le cn \lg n , \end{aligned}$$

Algoritmos Recursivos

- **Método de substituição – Exemplo**

Defina um limite superior sobre a recorrência abaixo:



$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n > 1. \end{cases} \quad (4.2)$$

As condições limite representam uma classe de detalhes que em geral ignoramos. Tendo em vista que o tempo de execução de um algoritmo sobre uma entrada de dimensões constantes é uma constante, as recorrências que surgem dos tempos de execução de algoritmos geralmente têm $T(n) = \Theta(1)$ para n suficientemente pequeno. Em consequência disso, por conveniência, em geral omitiremos declarações sobre as condições limite de recorrências e iremos supor que $T(n)$ é constante para n pequeno. Por exemplo, normalmente enunciamos a recorrência (4.1) como

$$T(n) = 2T(n/2) + \Theta(n) , \quad (4.3)$$

sem fornecer explicitamente valores para n pequeno. A razão é que, embora a mudança no valor de $T(1)$ altere a solução para a recorrência, normalmente a solução não muda por mais de um fator constante, e assim a ordem de crescimento não é alterada.

Quando enunciamos e resolvemos recorrências, com frequência omitimos pisos, tetos e condições limite. Seguimos em frente sem esses detalhes, e mais tarde definimos se eles têm importância ou não. Em geral, eles não têm importância, mas é importante saber quando têm. A experiência ajuda, e também alguns teoremas declarando que esses detalhes não afetam os limites assintóticos de muitas recorrências encontradas na análise de algoritmos (ver Teorema 4.1). Porém, neste capítulo, examinaremos alguns desses detalhes para mostrar os últimos métodos de solução de partes de recorrência.

4.1 O método de substituição

O método de substituição para resolver recorrências envolve duas etapas:

1. Pressupor a forma da solução.
2. Usar a indução matemática para encontrar as constantes e mostrar que a solução funciona.

O nome vem da substituição da resposta pressuposta para a função quando a hipótese indutiva é aplicada a valores menores. Esse método é eficiente, mas obviamente só pode ser aplicado em casos nos quais é fácil pressupor a forma da resposta.

O método de substituição pode ser usado para estabelecer limites superiores ou inferiores sobre uma recorrência. Como exemplo, vamos determinar um limite superior para a recorrência

Passo 2: realizando a indução matemática

T(1) = 1
T(1) <= c 1 lg 1 = 0
Logo T(1) = 1 != T(1) = 0

Algoritmos Recursivos